

14. Database Security

Database security and protection are receiving more attention and budget from organizations due to the steady increase in data breaches and the resultant regulations (see next chapter) designed to abate them. However, database security still requires more focus and effort. According to a 2005 report from Forrester Research, 75 percent of enterprises do not have a DBMS security plan.

Data Breaches

Data breaches continue to dominate both business and IT news, with bigger and uglier data breaches being regularly announced. A recent incident came from Epsilon, an e-mail marketing firm that sends billions of e-mails annually. The breach involved the exposure of e-mails for customers of many well-known brands, including the rewards programs of Ritz-Carlton and Marriott, banks such as Citibank and Capital One, and other companies such as Kroger and Walgreens.

How bad is this problem? The Privacy Rights Clearinghouse (www.privacyrights.org/data-breach) keeps track of every data breach that is reported. According to its research, more than 544 million records were breached between January 10, 2005, and early 2012—over the course of almost 3,000 separate data breach events.

Can we put a price tag on all of that unprotected and lost data? We can try.

A Forrester Research survey of companies that had experienced a data breach concluded that the average security breach can cost a company between \$90 and \$305 per lost record. But coming up with a precise figure can be difficult because of the additional, extenuating circumstances surrounding data breaches. The cost has to factor in such details as the expenses of legal fees, call centers, lost employee productivity, regulatory fines, customer losses, stock losses, and the nebulous cost of bad publicity.

The average security breach can cost a company between \$90 and \$305 per lost record.

Another research group, Ponemon Institute, conducts an annual study of

data breaches. According to its sixth annual U.S. Cost of a Data Breach Study, data breach costs increased to \$214 per compromised record in 2010. This is a significant increase from the 2006 report, which pegged the average cost per lost customer record at \$182.

If you are still skeptical, you can always roll your own numbers using the free Web-based data loss calculator provided by Darwin Professional Underwriters, Inc., at www.tech-404.com/calculator.html.

The obvious conclusion is that data breaches are costly, even at the low end of \$90 per record. Consider a typical data breach case. On February 27, 2008, Health Net Federal Services reported that thousands of doctors in 11 states had their personal information (including Social Security numbers) openly posted on a company Web site. According to the Privacy Rights Clearinghouse, the total number of records involved was 103,000. So what did that cost? At the low end, the cost was \$9.3 million, but at the high end it ballooned to over \$31.4 million. Using the Ponemon estimate, which is sort of in the middle, the cost ranged from \$18.7 million to \$22 million.

An additional interesting data point comes from the 2011 Data Breach Investigation Report, a study conducted by the Verizon RISK Team with cooperation from the U.S. Secret Service and the Dutch High Tech Crime Unit. According to this report, 76 percent of all data breached was compromised from servers. Furthermore, database servers yielded the majority of breached data.

So the cost of a data breach can be quite steep. And database servers are favorite sources for hackers to target. Thus, it makes sense to spend some time and money up-front to better secure and protect the data in your database systems.

Database Security Basics

The basic security and authorization approach taken by DBMS vendors to secure database access is that all database resources are controlled by the DBMS. No default authorizations are given to any user just because the user logs in to the DBMS. Therefore, for a user to be able to perform any DBMS operation or function, one of the following conditions must exist:

- The user has been granted the ability to perform that function or operation, or

- That operation or function has been granted generically to all users.

All database resources are controlled by the DBMS.

Using the security features of the DBMS, the DBA can set up the environment such that only certain users or certain application programs are allowed to perform certain operations on certain data within the database. Each user's function within the organization should determine the authorized level of database access. For example, only general-ledger programmers, batch jobs, and programs can access and modify the general-ledger databases. Different checks can be established for each type of access to each type of information, and different users can be assigned different access rights to different database objects.

The operational challenge of effectively administering database security arises because setting up and managing database authorization requires technical expertise and elevated privileges. Many aspects of database security require different utilities, system procedures, and commands to implement. When users require access to multiple databases, on multiple servers distributed across different physical locations, database security administration becomes quite complicated indeed. The commands must be repeated for each database, and there is no central repository for easily modifying and deleting user security settings on multiple databases simultaneously.

Although the DBA typically is responsible for administering database security, some organizations have transferred this task to a separate security administration function that controls all of the IT security for the company. However, even in many shops where security administration is a separate entity, database security is still handled by the DBA group because database security is managed differently from a typical IT authorization scenario.

When the security administration group handles security policies, this group usually relies on third-party security software such as IBM's RACF or Computer Associates' ACF2 and Top Secret. These products automate the security function and do not require the administrator to have elevated privileges to manage security policies. However, most of these security administration products run only on mainframes. Additionally, most IT security departments are understaffed and lack the technical DBMS expertise required to administer database security. Granting untrained security

personnel the privileges required to administer database security can result in accidental disruption of database service or performance problems. So, DBAs are forced to manage database security as a component of their job.

DBAs must manage database security as a component of their job.

At a high level, database security boils down to answering four questions:

- Who is it? (authentication)
- Who can do it? (authorization)
- Who can see it? (encryption)
- Who did it? (audit)

Strong authentication is the cornerstone of any security implementation plan. It is impossible to control authorization and track usage without it. Before authorization to use database resources can be granted, a login needs to be established for each user of the DBMS. Logins are sometimes referred to as *accounts*, or user IDs. The login will have a password associated with it such that only those who know the password can use the login ID. Some DBMSs use the operating system login ID and password as the DBMS login ID and password; others require an additional login ID and password to be created specifically for database access and security.

Strong authentication is the cornerstone of any security implementation plan.

When the DBMS controls the addition of logins, the DBA is required to provide certain information about the login when it is created. Typically, other than the actual login name or ID, the following information either can or must be provided:

- [*Password*](#)—the key phrase, word, or character string associated with the new login that must be provided by the user before access to the database is permitted
- *Default database*—the name of the database to which the user will initially be connected during login
- *Default language*—the default language assigned to the login when using the DBMS if multiple languages are supported

- *Name*—the actual full name of the user associated with this login
- *Additional details*—additional details about the user for which the login has been created: e-mail, phone number, office location, business unit, and so on (this is useful for documentation purposes)

Passwords should be changed regularly over time to make it difficult for hackers and miscreants to gain access to the DBMS surreptitiously. Refer to the sidebar “[Password Guidance](#)” for tips on creating useful passwords. As a DBA, you may decide to set up automated procedures—such as an e-mail notification system—to coerce users into changing their login password every month or so. Users who do not change their login password can be disabled until they call to complain. Of course, this adds to the workload of the DBA, but it does enhance the security of the DBMS.

Passwords should be changed regularly over time.

When a DBMS user no longer requires access to the DBMS, or leaves the company, the DBA should drop that user’s login from the system as soon as possible. However, this could become a complicated task—a login cannot be dropped if the person is currently using a database, or if the user owns any database objects.

For this reason it is wise to limit the database users who can create database objects to DBAs only, especially in a production environment.

Limit the database users who can create database objects to DBAs only.

In lieu of dropping a login, the DBMS may provide an option to lock the login. Locking a login prohibits the user from accessing the DBMS, but it does not actually drop the login from the system. The login can subsequently be unlocked, thereby enabling access to the server. Such a process is very useful if you simply wish to prohibit access, say, for those users who have not recently changed their login password.

Keep in mind that logins that are dropped must be created anew if they are ever required again. For this reason, follow these rules of thumb regarding login administration:

- Lock logins that may need to be reactivated.
- Drop logins that will never need to be reactivated.

Password Guidance

As a DBA, you are responsible for securing the DBMS and granting data access to authorized users. One way of assuring proper DBMS usage is to develop and disseminate tips on creating useful passwords. A password that is useful and proper will be difficult to guess.

If passwords are too simplistic or too related to some aspect of the person using the password, unscrupulous individuals may be able to guess the password and use it to surreptitiously access the database.

The following guidelines should be followed for proper password creation:

1. Avoid passwords that are too short. Each password should be at least six characters long, more if possible.
2. Each password should consist of at least a combination of alphabetic characters and numeric characters. Using other allowable symbols makes the password harder to guess.
3. Avoid creating a password that is a complete word (in either the native language of the user or any foreign language).
4. Do not embed personal statistics in the password. Street addresses, Social Security numbers, phone numbers, and the like are easily guessed and do not belong in passwords.
5. Consider concatenating two unrelated words with a symbol or number between them. For example, “toe3star” is a viable password.

Use mnemonic devices to help you remember passwords. For example, use a sentence such as “Moondance by Van Morrison is my favorite album” to remember that your password is “mbvmimfa” (the first letters of each word in the sentence). However, do not make the sentence too obvious—for example, using “My name is Craig S. Mullins” to remember “mnicsm” is not a good idea because it might be quite easily guessed.

Other common and weak password archetypes include sports teams and sports celebrities (like “ChiBulls99” or “TigerWoods”), naughty words (curse words/anatomy), and names of family members. In general, if you think it can be easily guessed, avoid using it.

As a DBA, you should work with your organization’s security

administration team to create guidelines such as these and distribute them to your database users.

Some DBMSs provide additional controls and parameters on logins and passwords. For example, some DBMSs provide profile parameters for passwords that can be used to limit the following:

- Number of failed login attempts before the account is locked
- Number of days a password is valid, the grace period for changing an expired password
- Number of days the account can remain locked when the password expires
- Reusability of passwords (number of days before a password can be reused and the maximum number of times a password can be reused)

When such controls are available, be sure to use them when doling out login accounts to better secure the DBMS environment.

Keep in mind, though, that each DBMS is different, and there may be no capability to force a user to periodically change a password. If a password is never changed, the likelihood of it being compromised increases over time. If the database is able to force a periodic password change, it is often limited in its ability to completely enforce corporate password standards that reduce the risk of the password being guessed. These standards usually include minimum length and alphanumeric requirements. Most DBMSs do not even provide a simple interface by which end users can change their own password. The problem is compounded if users have accounts on multiple databases across multiple servers.

Enforce corporate password standards.

Database Users

In addition to a login account, which is required to access the DBMS, some database systems require an additional account in order to use specific databases. In this situation, a user name is created for the login account and is attached to each database required by the user. As shown in [Figure 14.1](#), the following accounts may be required:

- A *login*, sometimes called an *account*, is used to access the DBMS, or

database server. For this reason it is sometimes also known as a *server user ID*, or SUID.

- A *user name* is sometimes referred to as a *database ID*. The user name is associated with the login account. Users are required by some DBMS implementations to be set up with a database user name in order to access each database.

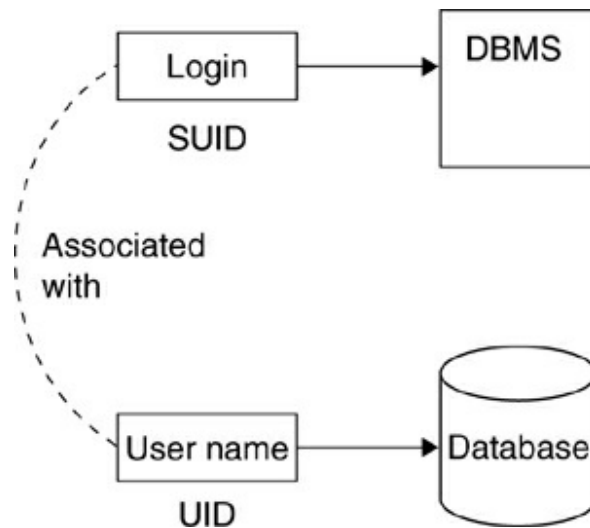


Figure 14.1. DBMS and database logins

Guest usage of a database is permitted by configuring a special user name that permits users to access the database as a GUEST. Adding a GUEST user for a database allows any login with the special user name to access the database.

Granting and Revoking Authority

The DBA controls database security and authorization using Data Control Language, or DCL. DCL is one of three subtypes of SQL. (The other two are DDL and DML.) DCL statements are used to control which users have access to which objects and commands. These statements are the manner in which database security is enacted. DCL statements comprise two basic types:

- *GRANT* assigns a permission to a database user.
- *REVOKE* removes a permission from a database user.

DCL statements comprise two basic types: GRANT and REVOKE.

The GRANT statement is issued with two accompanying lists: a list of privileges to be assigned to a list of users. To use the GRANT statement, the user must be the owner of the database object, have been granted high-level group authority, or have been given the WITH GRANT OPTION when the privilege was granted.

The WITH GRANT OPTION allows a user to pass the authority to grant privileges along to others. Generally, the use of this clause depends on whether an installation practices centralized or decentralized administration of privileges.

- *Decentralized administration* is generally easier to establish, but more difficult to control. As more and more users obtain the authority to grant privileges, the scope of authority is widened and becomes unwieldy.
- *Centralized administration* is generally easier to administer but places a burden on the centralized administrator as the sole arbiter of privileges within the environment.

Avoid issuing GRANT and REVOKE statements from within an application program. Ideally, an individual who understands the security needs of the organization—usually the DBA—grants database authorization. Furthermore, application programs designed to grant database privileges must be executed by a user who has the appropriate authority to issue the GRANTS and REVOKEs coded into the application program. This could create a loophole in your database security infrastructure.

Some DBMSs provide additional functionality for conferring or prohibiting database authority. For example, Microsoft SQL Server includes the DENY statement to specifically deny privileges to a specific user.

Types of Privileges

There are different types of privileges that can be granted and revoked from database users. Every DBMS provides certain basic types of privileges, such as the ability to access data, to create database objects, and to perform system functions. Each DBMS will also have additional types of privileges, depending on the features it supports.

The following types of privileges are commonly provided by modern DBMSs:

- [Table](#): to control who can access and modify the data within tables
- *Database object*: to control who can create new database objects and drop existing database objects
- *System*: to control who can perform certain types of system-wide activities
- *Program*: to control who can create, modify, and use database programs
- [Stored procedure](#): to control who can execute specific functions and stored procedures

Granting Table Privileges

Table privileges are granted to enable users to access tables, views, and columns within tables and views. The following privileges can be granted for tables and views:

- SELECT: to enable the user to select from this table/view
- INSERT: to enable the user to insert rows into this table/view
- UPDATE: to enable the user to update this table/view
- DELETE: to enable the user to delete rows from this table/view
- ALL: to enable the user to select, insert, update, and delete using this table/view

For example, to enable user7 to delete rows from the Titles table, the following statement can be issued:

```
GRANT DELETE on Titles to user7;
```

Some table privileges can be specified at the column level. Doing so can be desirable when certain users must be permitted to modify specific columns of a table but not other columns. The SELECT and UPDATE privileges can be granted for specific columns. For example, to enable user7 to update only the au_id column in the Titles table, the following statement can be issued:

```
GRANT UPDATE on Titles (au_id) to user7;
```

Typically, the DBA will grant table privileges to programmers in a test environment for development purposes. Additionally, programmers and end users may require table privileges on production tables for certain tasks. However, most production access should be controlled using program and

stored procedure privileges, instead of direct table privileges.

Granting Database Object Privileges

Database object privileges control which users have permission to create database structures. The actual privileges that can be granted will depend on the DBMS and the types of database objects supported. Generally, the DBMS will provide options to grant CREATE privileges on each type of database object, including databases, tablespaces, tables, indexes, triggers, defaults, and user-defined data types.

Database object privileges control which users have permission to create database structures.

For example, to enable user5 and user9 to create tables and indexes, the following statement can be issued:

```
GRANT  CREATE table,  
        CREATE index  
TO user5,  
   user9;
```

The ability to create database objects is usually reserved for DBAs. If these privileges are granted to others, the number of existing database objects can be quite difficult to control. Furthermore, it becomes very difficult to track which database objects are really being used, and which were created and then abandoned. For these reasons, DBAs should keep this authority to themselves, with only rare exceptions perhaps for SAs or very skilled developers.

The ability to create database objects is usually reserved for DBAs.

Granting System Privileges

System privileges control which users can use certain DBMS features and execute certain DBMS commands. The system privileges available will vary from DBMS to DBMS but may include the ability to archive database logs, shut down and restart the database server, start traces for monitoring, manage storage, and manage database caches.

System privileges cannot be granted at the database level. System privileges are granted at a system-wide level across the DBMS. For example,

to enable user6 to start performance traces, the following statement can be issued:

```
GRANT TRACE
TO      user6;
```

System privileges cannot be granted at the database level.

System privileges should be granted with care and should usually be reserved for the DBA and SA.

Granting Program and Procedure Privileges

Granting the EXECUTE privilege gives the user permission to execute a program or a stored procedure. For example, to enable user1 and user9 to execute the stored procedure named proc1, the following statement can be issued:

```
GRANT EXECUTE on proc1
TO      user1, user9;
```

Granting privileges to users on programs and procedures is easier to control than granting privileges on individual tables and columns. The procedural logic in the program and procedure controls which specific tables and columns can be modified. Furthermore, the DBA can better maintain the integrity of production data if the only way it can be changed is programmatically.

Granting to PUBLIC

As an alternative to granting access to a database user, the DBA can choose to grant a particular authorization to PUBLIC. When authorization is granted to PUBLIC, the DBMS will allow anyone who can log in to the DBMS that particular authority. Grants made to PUBLIC cannot be given with the WITH GRANT OPTION, as everyone is in PUBLIC.

For example, to grant everyone the authority to delete rows from the titles table, the following statement can be issued:

```
GRANT DELETE on titles to PUBLIC;
```

Administering security can be a complex duty. Using the PUBLIC authority to permit blanket access to certain database objects and resources often appears easier than specifically controlling access by user. However,

DBAs should exercise caution when granting any privileges to PUBLIC.

Exercise caution when granting any privileges to PUBLIC.

Whenever a privilege is granted to PUBLIC, the DBA loses control over that database object or resource—anyone can access or use the object or resource as specified by the GRANT statement. Inevitably, users will abuse PUBLIC resources, and gaining control over the resource again will be quite difficult.

Reserve usage of PUBLIC authority for those few database objects and resources that should be available to everyone. Alternatively, PUBLIC authority can be a useful shortcut if another security mechanism is in place. For example, you could grant program authority to PUBLIC for transactions that are run under a transaction processor and use the security facilities of the transaction processor to control access.

PUBLIC Authority and the System Catalog

A common dilemma faced by DBAs is whether or not to grant PUBLIC access to the system catalog tables. On the one hand, the metadata contained in the system catalog is useful information that will be needed by most DBAs, developers, and analysts as they create and maintain applications that access production databases. On the other hand, the system catalog frequently contains sensitive information that can be exploited by hackers. For example, system catalog tables that contain authorization details can be queried to learn who has access to which tables. These tables typically contain user ID and database login information, too. Furthermore, being able to query the system catalog exposes the names of all the tables that have been created in the system, making it easier for a hacker to attempt to access data (or to attempt a SQL injection attack).

The bottom line is that you should refrain from granting blanket PUBLIC access to the system catalog tables.

Refrain from granting blanket PUBLIC access to the system catalog tables.

Revoking Privileges

The REVOKE statement is used to remove privileges that were previously granted. The syntax for REVOKE is the “flip side” of the GRANT syntax.

Additionally, privileges will be automatically revoked by the DBMS when a database object is dropped.

For example, to revoke the ability to update the `au_id` column of the `titles` table from `user7`:

```
REVOKE UPDATE on titles (au_id) from user7;
```

Revoking a `PUBLIC` privilege will not remove that privilege from any user to whom it was granted in a separate `GRANT` statement.

Cascading REVOKEs

When privileges are revoked, the DBMS must decide whether additional revokes are necessary, based on the privileges being revoked. When one revoke causes the DBMS to revoke additional related privileges, it is called *cascading REVOKEs*. Consider the authority hierarchy depicted in [Figure 14.2](#). Joe has been granted the ability to grant a privilege, say *X*, to others. He grants *X* to Pete and Phil with the `GRANT` option. Pete in turn grants *X* to Bruce. Joe also grants *X* to Don, but without the `GRANT` option.

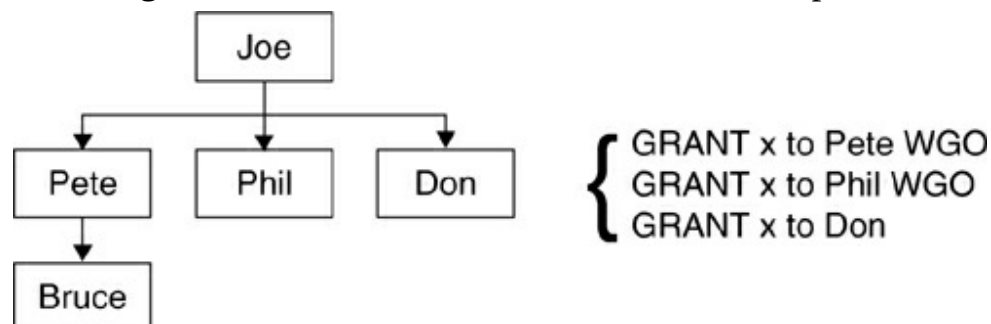


Figure 14.2. Cascading REVOKEs

Now let's investigate the impact of cascading `REVOKEs` by outlining what happens if we revoke the *X* privilege from Joe. In this case, not only will Joe no longer have the *X* privilege, but the DBMS will also remove the authority from Pete, Phil, and Don. Furthermore, because Phil's *X* privilege was revoked, the effect of the revoke will cascade to Bruce, too.

To minimize the impact of cascading `REVOKEs`, avoid granting privileges using the `WITH GRANT OPTION`. The fewer users who can grant subsequent privileges, the easier it is to manage and administer a viable DBMS security infrastructure.

Some DBMS products provide an option of the `REVOKE` statement

enabling authority to be revoked without cascading.

Chronology and Revokes

The timing of a GRANT or REVOKE statement may have a bearing on its impact. For example, in some DBMSs it is possible to grant a privilege to all users except a specific user by issuing the following statements:

```
GRANT DELETE on titles to public;  
COMMIT;  
REVOKE DELETE on titles from userx;
```

The timing of a GRANT or REVOKE statement may have a bearing on its impact.

The first statement grants everyone the authority to delete data from the titles table. Because the REVOKE statement is issued after the GRANT to public, the individual userx is barred from deleting data from this table. Some DBMSs (e.g., DB2) do not permit such exclusions, because the PUBLIC authority will override any revokes; other DBMSs (e.g., Microsoft SQL Server) do permit such exclusions.

The DBA needs to understand exactly how GRANTs and REVOKEs work for each DBMS being administered in order to properly manage privileges on database objects and resources.

Label-Based Access Control

As database systems and applications become more sophisticated, the need for low-level access control to the data becomes more critical. A growing number of DBMSs offer label-based access control (LBAC) to ensure that each piece of data is secured such that only authorized users can perform authorized functions. With LBAC it is possible to support applications that need a more granular security scheme. LBAC can be set up to specify who can read and modify data in individual rows and/or columns.

LBAC provides for a more granular security scheme, specifying who can read and modify data in individual rows and/or columns.

For example, you might want to set up an authorization scenario such that employees can see their own data but no one else's. Or, your authorization

needs might dictate that each employee's immediate manager is able to see his payroll information as well as all of his employees' data, and so on up through the org chart. Setting up such a security scheme is virtually impossible without LBAC.

An administrator must configure the LBAC system by creating security label components, which are database objects used to represent the conditions determining whether a user can access a piece of data. A security policy, composed of one or more security label components, is used to describe the criteria for determining who has access to what data. The security administrator defines the policy by creating security labels that are composed of security label components.

Once created, a security label can be associated with individual columns and rows in a table to protect the data held there. An administrator allows users access to protected data by granting them security labels. When a user tries to access protected data, that user's security label is compared to the security label protecting the data. The protecting label will block some security labels but not others.

Any individual user can hold security labels for multiple security policies. For any given security policy, however, a user can hold at most one label for read access and one label for write access. A security administrator can also grant exemptions to users. An exemption allows users to access protected data that their security labels might otherwise prevent them from accessing. Together, security labels and exemptions are called LBAC credentials.

Any attempted access to a protected column when the LBAC credentials do not permit that access will fail. If users try to read protected rows not allowed by their LBAC credentials, the DBMS simply acts as if those rows do not exist. Even aggregate functions, such as COUNT(*), will ignore rows when the LBAC credentials do not allow read access. This is important because sometimes even having knowledge that the data exists (without being able to access it) must be protected.

To use LBAC, you will need to add a specially named column to act as the security label. The security label column is matched with the multilevel security hierarchy set up by the administrator. For example, you might want to set up a hierarchy using colors, as shown in [Figure 14.3](#). Of course, the hierarchy need not be so complex—you might choose to use something simpler, such as TOP SECRET, SECRET, and UNCLASSIFIED.

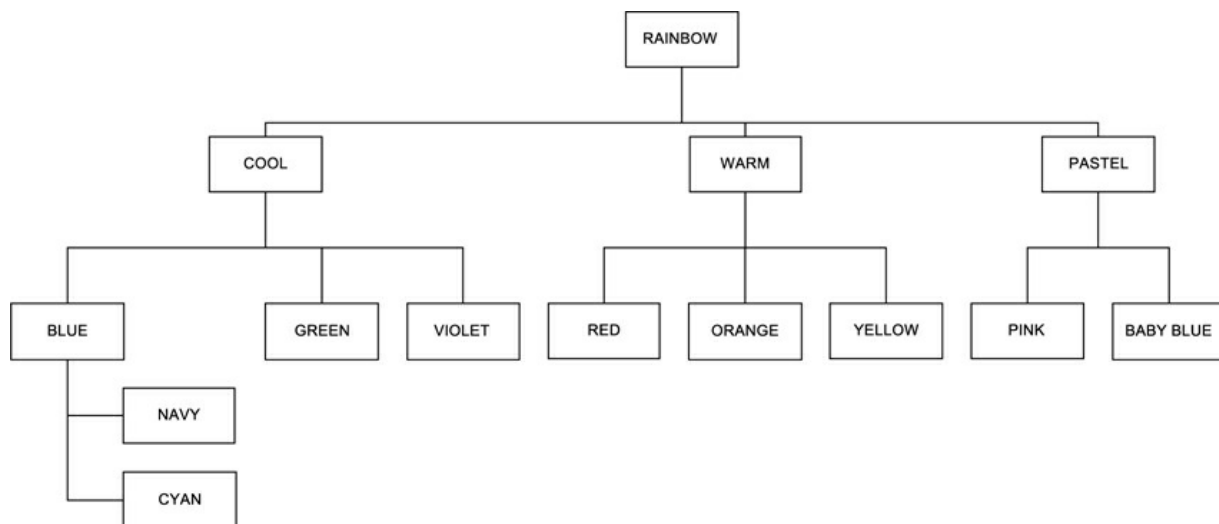


Figure 14.3. Sample LBAC security hierarchy

Consult your DBMS documentation for where and how to establish this hierarchy. For example, with DB2 for z/OS the hierarchy is established in security software, such as RACF. At the top of the hierarchy in the example ([Figure 14.3](#)), the RAINBOW security label includes everything—all colors are included in the rainbow. Middle levels of the hierarchy that represent additional security groupings can be created. In this case we have a second level of COOL, WARM, and PASTEL. Under COOL, we see the cool colors, BLUE, GREEN, and VIOLET. Even further categorization is possible as shown with the various shades of BLUE. And so on throughout the hierarchy. Such a hierarchy provides great flexibility for assigning various levels of security to data in your tables—and thereby appropriately securing the data from unauthorized user access.

Assigning a security label column in your table, and populating that column with the appropriate specification from the hierarchy, makes it possible to control whether or not an individual user can access that particular row. So assigning a row to the security label of RAINBOW makes the data accessible to anyone, whereas assigning the row to NAVY would be much more particular and prohibitive. Users with authority to access RAINBOW can access data because all colors fall under the rainbow. BLUE data can access any row associated with BLUE, or any subordinate blue shade (NAVY and CYAN).

Assigning a security label makes it possible to control whether

or not an individual user can access a particular row.

LBAC offers a very powerful security setup. Such strict security controls are common requirements for government databases, for example.

Security Reporting

Once granted, the DBA will need to monitor and report on the privileges held by users. Database security is maintained in the system catalog. The DBA can use SQL to retrieve the needed information from the appropriate system catalog tables. Alternatively, some DBMSs provide views and system-stored procedures that simplify the retrieval of database security.

As a further consideration, though, be sure to adequately protect the security of the system catalog, especially within the production system. Only the DBA, SA, and security administrator require access to the database security information stored in the system catalog.

Be sure to adequately protect the security of the system catalog.

User security requirements and expectations tend to evolve over time. As new applications are added and business requirements change, database security will need to change. Security reviews should be performed on a regular basis to ensure that database security as implemented continues to match the current user requirements. Reports from the system catalog tables can be used to provide the input for such reviews.

Authorization Roles and Groups

In addition to granting privileges to individual users, the DBMS may provide the capability to assign users

- Specific privileges to a role, which is then granted to others, or
- Specific built-in groups of privileges

Of course, the terminology is not strict among the major DBMSs. Some DBMSs refer to roles as groups, and vice versa. As a DBA, you will need to understand how each DBMS you manage implements roles and groups and how each of these features can be used to simplify database security administration.

Roles

Once defined, a [role](#) can be used to grant one or more preassigned privileges to a user. A role is essentially a collection of privileges. The DBA can create a role and assign certain privileges to that role. Then the role can be assigned to one or more users. The administration of database security is simplified in this way. For example, consider the following sequence of statements:

The DBA can create a role and assign certain privileges to that role.

[Click here to view code image](#)

```
CREATE role MANAGER;  
COMMIT;  
GRANT select, insert, update, delete on employee to  
MANAGER;  
GRANT select, insert, update, delete on job_title to  
MANAGER;  
GRANT execute on payroll to MANAGER;  
COMMIT;  
GRANT MANAGER to user1;  
COMMIT;
```

This script creates a new role named MANAGER, grants privileges on certain tables and procedures to the role, and then assigns user1 the MANAGER role. Additional users can be assigned the MANAGER role, and the DBA will not need to remember to issue each of the individual GRANT statements, because they have already been assigned to the MANAGER role.

Groups

Group-level authority is similar to roles. However, each DBMS provides built-in groups that cannot be changed. Each DBMS implements group-level database security in different ways and with different group names and privileges. There are some similarities across DBMSs, though. The following groups are common among the major DBMSs:

- *System administrator*. Sometimes abbreviated SA or SYSADM, the system administrator group is the most powerful within the DBMS. A user granted SA-level authority typically can execute all database commands and access all databases and objects. The system administrator is usually responsible for installing the DBMS and is

viewed as the owner of system resources and system catalog tables.

- *[Database administrator](#)*. Sometimes abbreviated as DBADM or DBA, the database administrator group gives all privileges over a specific database, plus the ability to access, but not modify, data in tables within that database. Users assigned DBA-level authority can drop and alter any objects within the database (tablespaces, tables, and indexes).
- *Database maintenance*. Sometimes abbreviated as DBMAINT, the database maintenance group includes the specific database privileges for maintaining database objects (such as the ability to run utilities and issue commands). Like the DBA group, the DBMAINT-level privilege is granted on a database-by-database basis.
- *Security administrator*. The security administrator role has the privilege set permitting the granting and revoking of database security across the DBMS. Any database security-related activities can be performed by the security administrator, including login and password administration, auditing, security configuration, as well as grants and revokes. Another common name for the security administrator role is SSO.
- *Operations control*. Sometimes referred to as OPER or SYSOPR, the operations control role has the authority to perform operational database tasks such as backup and recovery, or terminating runaway tasks.

Each DBMS provides built-in groups that cannot be changed.

Limit the Number of SA Users

A single organization should limit the number of users who are assigned the SA role or group-level authority.¹ A user with SA capabilities is very powerful. Only corporate DBAs and systems programmers should be granted this level of authority. End users, managers, and application development personnel do not need SA authority to do their jobs.

A user with SA capabilities is very powerful.

Group-Level Security and Cascading REVOKEs

Depending on the group, some users who have been assigned group-level privileges can grant privileges to other users. If the group-level authority is revoked from such a user, any privileges that user granted will also be

revoked. This is similar to the cascading REVOKEs that occur as a result of the WITH GRANT option.

Before revoking a group-level authorization from a user, be sure to ascertain the impact of cascading REVOKEs, and be prepared to reapply the required privileges that will be removed due to the cascading effect.

Other Database Security Mechanisms

Modern relational DBMS products support many capabilities and qualities that can aid in securing data. Some of these capabilities are not primarily security features. For example, views and stored procedures can be used for security purposes, even though that is not their main purpose.

Using Views for Security

Most database security is performed using the native security of the DBMS. However, it is possible to simplify some aspects of database security by creating views to protect your data.

Your organization has deployed an employee table that houses pertinent information about all employees. Columns within the table exist to store the employee's first and last name, middle initial, address, telephone number, salary, and so on. Granting the SELECT privilege on the employee table to a group of users can cause a security problem. While application security is maintained with this scenario, personal security is not because the user could access the personal details, including salary information, of fellow employees.

A view could be created that omits the sensitive information from the employee table. When the view is created without the sensitive columns, users can be granted the SELECT privilege on the view and will be able to access employee information that is deemed appropriate. For example:

A view can be created that omits sensitive information.

[Click here to view code image](#)

```
CREATE view emp_all
AS
SELECT first_name, last_name, middle_initial,
       street_address, state, zip_code
FROM   employee;
```

This simple example shows a view that specifies only certain columns from the base table. Once the view has been created and the user has been granted the SELECT privilege on the view, only the information specified in the view can be retrieved. Eliminating columns from a base table to create a view is referred to as *vertical restriction*.

Of course, the definition of *sensitive* will vary from organization to organization. In our example, the salary and telephone information were removed. It is quite simple to understand why salary is sensitive, but what about telephone number? And if telephone number is sensitive, perhaps the employee's address should be as well. Views allow you to easily specify the column-level security deemed necessary for your organization.

Vertical restriction using views is an alternative to specifying columns when granting table privileges. It also can be easier to implement and administer.

Views can also be used to provide row-level security based on the content of data. This is called *horizontal restriction* and is implemented by coding the appropriate WHERE clauses into the view. For example:

[Click here to view code image](#)

```
CREATE view emp_dept20
AS
SELECT first_name, last_name, middle_initial,
       street_address, state, zip_code
FROM   employee
WHERE  deptno = 20;
```

When users select from the view, only rows that match the predicate will be returned. This view will return only those employees who work in department 20. When users modify rows of the view, if the WITH CHECK OPTION has been specified, the predicates will ensure that values cannot be updated or inserted out of the range. Additionally, rows that do not match the predicate cannot be deleted using the view when WITH CHECK OPTION is specified.

Using Stored Procedures for Security

Stored procedures can be used to provide an additional level of security. The privilege to execute a stored procedure must be explicitly granted or revoked, regardless of the security implemented on the underlying base tables.

The privilege to execute a stored procedure must be explicitly granted or revoked.

Stored procedures can be coded that access only row- and/or column-level subsets of data. The ability to execute these stored procedures can then be granted to users. If no privileges on the underlying base tables are granted, users will be able to access the data only by executing the stored procedure, thereby providing the requisite security.

In addition to providing a level of security, this method can provide better performance if the algorithms in the procedure are coded properly.

Logic-Oriented Security

Sometimes it is necessary to implement security based on an algorithm. For example, what happens if only a subset of users can access a specific table during a specific time of day? This criterion can be coded into the stored procedure. Whenever the stored procedure is executed, it checks the user and the time of day before permitting access.

Some DBMS products offer special functionality for database security. See the sidebar “[Oracle Virtual Private Database](#)” for an example of such functionality.

Encryption

Another security feature for data protection is encryption. Data encryption is a process whereby data is transformed using an algorithm to make it unreadable to anyone without the decryption key. The general idea is to make the effort of decrypting so difficult as to outweigh the advantage to a hacker of accessing the unauthorized data. The general flow of encryption and decryption is depicted in [Figure 14.4](#).

Encryption transforms data, rendering it unreadable to anyone without the decryption key.

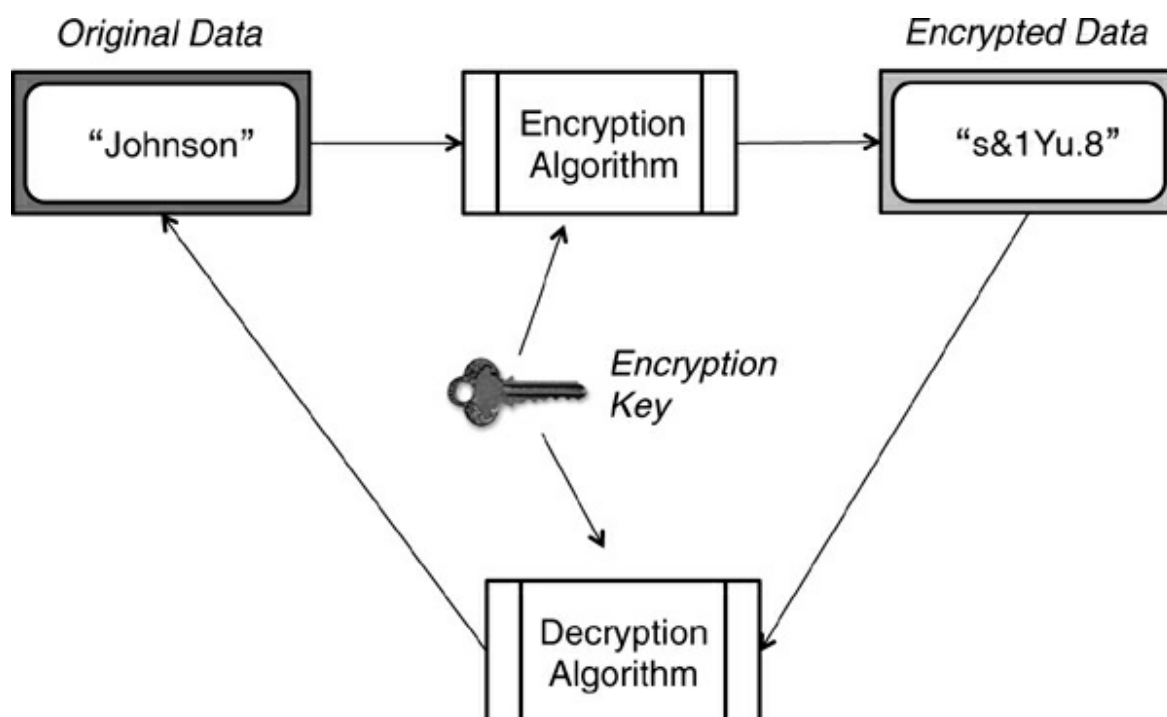


Figure 14.4. Encryption and decryption

Oracle Virtual Private Database

Oracle provides row-level access control through its Virtual Private Database (VPD) technology. VPD is enabled by associating one or more security policies with tables or views. When the table is accessed, either directly or indirectly, the database will consult a function that implements the policy. The policy is basically a SQL predicate (or WHERE clause) that the database appends to the user's SQL statement. This dynamically modifies the user's data access.

With VPD, a user is able to retrieve and manipulate only data that matches the WHERE clause in the policy. In essence, this works something like creating a dynamic view (using WITH CHECK OPTION) that is always applied and enforced.

Encryption has been used by governments and military organizations for years to enable secret data transmission and communication. Because of the added data protection and regulatory requirements of late, encryption is being more commonly used in many business systems and applications.

There are two types of situations where data encryption can be deployed: data in transit and data at rest. In a database context, data “at rest” encryption protects data stored in the database, whereas data “in transit” encryption is used for data being transferred over a network.

Data at Rest Encryption

Encrypting data at rest is undertaken to prohibit “behind the scenes” snooping for information. Consider, for example, a database containing a top-secret military plan. Of course, this data can, and should, be protected using traditional database security and authorization methods, such as granting access to only those with the proper clearance. Additional precaution can be taken by using LBAC techniques if the database contains data needing varying levels of clearance requirements. But what if the data is accessed outside the control of the DBMS?

When the data at rest is encrypted, even if a hacker surreptitiously gains access to the data behind the scenes, without the decryption key the data will be meaningless.

Without the decryption key the data will be meaningless.

Other examples of data at rest encryption in database systems include encrypting backup data and encrypting user IDs and passwords.

Data at rest encryption most commonly is supported by using SQL functions or through an add-on encryption product. Oracle supports transparent data encryption through the use of a “wallet” (see the sidebar).

Data in Transit Encryption

Encrypting data in transit is undertaken to prohibit network packet sniffing. If the data is encrypted before it is sent over the network and decrypted upon receipt at its destination, it is protected along its journey. Anyone nefariously attempting to access the data en route will receive only encrypted data. And again, without the decryption key, the data cannot be deciphered.

Data in transit encryption most commonly is supported using DBMS system parameters and commands or through an add-on encryption product.

Encryption Techniques

There are various encryption techniques and algorithms supported by modern

DBMSs. Among the more common are Data Encryption Standard (DES), Triple DES (TDES), and Advanced Encryption Standard (AES)-128 bit data encryption/decryption, as well as Secure Hash Algorithm (SHA-1) and SHA-256 hashing.

Oracle Transparent Data Encryption

Transparent data encryption enables you to encrypt individual table columns or an entire tablespace. When a user inserts data into an encrypted column, transparent data encryption automatically encrypts the data. When users select the column, the data is automatically decrypted.

Data is encrypted using a wallet, which is an operating system file located outside the database. The database uses the wallet to store the encryption key. The wallet is created using the ALTER SYSTEM command (or within Oracle Enterprise Manager). The wallet itself is encrypted and requires a password as its encryption key. To access the contents of the wallet you must know the password. After the wallet is created, you must open the wallet using the password so that the database can access the master encryption key. The location of the wallet is specified in the sqlnet.ora file.

So, when a user enters data, Oracle retrieves the master key from the wallet, decrypts the encryption key using the master key, uses the encryption key to encrypt the data entered by the user, and stores the encrypted data in the database. When a user accesses data, the process is similar. Oracle decrypts the data and displays it in clear text format.

SQL Injection

Another aspect of database security is designing your applications properly in order to avoid SQL injection attacks. SQL injection is a form of Web hacking whereby SQL statements are specified in the fields of a Web form to cause a poorly designed Web application to dump database content to the attacker. SQL injection is a favorite technique of Web hackers, and stories abound in the news of the technique being used for nefarious purposes (see the sidebar titled “[Examples of SQL Injection Attacks](#)”).

SQL injection is a form of Web hacking whereby SQL

statements are specified in a Web form to expose data to the attacker.

In order for SQL injection to succeed, the application software used by the Web site must be vulnerable to an injection attack. SQL injection relies upon programs that do not adequately filter for string literal escape characters embedded in SQL statements or where user input is not strongly typed. So instead of data being input into a form, SQL statements are supplied. The SQL is “injected” from the Web form into the database, causing it to be executed and to access (or even modify) unintended data.

Perhaps it is easiest to comprehend SQL injection by example. Consider a Web-based application using dynamic SQL. The Web site requires users to log in with their e-mail address and a password. Almost all sites of this type also offer an option to retrieve your password by supplying your e-mail address. Perhaps the SQL looks something like this:

```
SELECT userid, password
FROM   uid_pwd_table
WHERE  field = '$EMAIL';
```

Examples of SQL Injection Attacks

There are many examples of SQL injection being used to attack popular Web sites. For example:

On January 13, 2006, Russian hackers broke into a Rhode Island government Web site and obtained credit card data from residents who had done business online with state agencies (www.xiom.com/whid-2006-3).

On June 29, 2007, a hacker defaced the Microsoft Web site in the United Kingdom using a SQL injection attack (www.cgisecurity.com/2007/06/hacker-defaces.html).

In January 2008, tens of thousands of PCs were infected by an automated SQL injection attack that exploited a vulnerability in application code that accesses Microsoft SQL Server databases (www.pcworld.com/businesscenter/article/146048/mass_sql_injection_atta

On September 19, 2010, during the Swedish election, a voter attempted an injection by hand-writing SQL statements as part of a write-in vote (<http://alicebobandmallory.com/articles/2010/09/23/did->

[little-bobby-tables-migrate-to-sweden](#)).

In June 2011, the Public Broadcasting System (PBS) was hacked, through use of SQL injection (<http://blog.imperva.com/2011/05/pbs-breached-how-hackers-probably-did-it.html>).

In September 2011, Turkish hackers accessed NetNames DNS records and changed entries redirecting users (of the *Telegraph*, the *Register*, the *National Geographic*, and others) to a site set up by them (www.zone-h.org/news/id/4741).

Other Web sites attacked by SQL injection in 2011 included Barracuda Networks, mysql.com, Lady Gaga's Web site, Nokia's developer site, and Canon, among other prominent organizations.

The variable \$EMAIL represents the input from the form on the Web site. A savvy hacker can attempt a SQL injection attack by entering

```
anything' OR '1'='1
```

If the application does not check the input properly, the injection causes the SQL to now look like this:

```
SELECT userid, password
FROM   uid_pwd_table
WHERE  field = 'anything' OR '1'='1';
```

This will cause a complete dump of every user ID and password in the database because the OR '1'='1' component will always evaluate to TRUE. It does not matter what the first part of the injection was; it could be anything, because the second part of the injection gives the hacker everything in the table.

Another form of SQL injection relies upon improper typing, for example, not checking whether data that should be numeric is actually numeric. Consider, for example:

[Click here to view code image](#)

```
statement := "SELECT * FROM userinfo WHERE id = " +
in_var + ";"
```

In this case, the SQL is being built into the statement variable; the in_var is the variable used to supply the input. Let's assume that the id column is

numeric. However, if the program does not check the data type of the `in_var` variable to ensure that numeric data is supplied, SQL injection can occur. For example, instead of just supplying a numeric value, the hacker can supply something like this:

```
4;DROP TABLE customer
```

If this SQL statement is built and executed, the customer table (if one exists) will be dropped from the database.

SQL Injection Prevention

Using well-designed query language interpreters and coding applications appropriately can prevent SQL injection attacks. When possible, use static SQL (see the next section for more details).

Always validate user input by testing type, length, format, and range. The program should make absolutely no assumptions about the size, type, or content of the data that is received. Test the size and data type of input and enforce appropriate limits. Doing so can help to prevent buffer overruns. Test the content of string variables and allow only expected values to be processed. Any input that contains binary data, escape sequences, and comment characters should be summarily rejected.

Always validate user input by testing type, length, format, and range.

Avoid concatenating user input that has not been validated. String concatenation is the primary point of entry for SQL injection attacks. Furthermore, consider using stored procedures to validate user input.

Analyze input and reject anything that contains special characters such as the semicolon (;), the string delimiter ('), comment delimiters (--, /*...*/), V\$ (the beginning of Oracle DBA views), and xp_ (the beginning of SQL Server catalog stored procedures).

For Microsoft SQL Server users, beware of the Transact-SQL procedure `xp_cmdshell`. This procedure spawns a Windows command shell and passes in a string for execution. So it is entirely possible for a SQL injection attack not only to steal or damage data, but to upload and execute its own code, probe the network, and even launch attacks against other sites from the victim of the SQL injection. Use of `xp_cmdshell` is disabled by default, and it is a

good idea to keep it that way to thwart malicious users from attempting to elevate their privileges using the command.

With foreknowledge of SQL injection techniques and proper development procedures, all SQL injection attacks can be prevented.

Static versus Dynamic SQL

Using static SQL instead of dynamic SQL can improve the security of your database applications and data. Static SQL is hard-coded into the application and cannot be changed at run time. Dynamic SQL is flexible and can change at run time. When the SQL can change at run time, a sufficiently motivated and skilled hacker can potentially change the SQL or potentially deploy a SQL injection attack to gain access to unauthorized data.

Using static SQL instead of dynamic SQL can improve security.

Static SQL is common in mainframe DB2 applications, but not so much for other platforms and database systems. The DB2 BIND command “hardens” the SQL and optimizes access to the data. See the sidebar on the DB2 BIND command for additional details.

If you are using a CLI, such as ODBC or JDBC, to access the database, static SQL is probably not an option. However, you can use stored procedures to gain a measure of protection similar to that of static SQL.

The DB2 BIND Command

The BIND command can be thought of as a compiler for SQL statements. In general, the BIND command reads SQL statements embedded in the application program and produces an executable access path to the data as directed by the SQL statements being bound. Once the SQL is statically bound, it cannot be changed without undergoing the BIND process again.

Binding performs many functions to create packages that access the requested data, including reading the SQL statements in the program, syntax checking, metadata validation, authorization validation, and optimization.

An added security feature of DB2 binding is provided with the ENABLE and DISABLE options, which can be specified to control the environment in which the plan or package being bound can be executed.

ENABLE ensures that the program operates in only the enabled environments. DISABLE permits execution of programs everywhere except for those environments explicitly disabled. ENABLE and DISABLE are mutually exclusive parameters; only one can be used per program package. For example, if a program is bound specifying ENABLE(CICS), only the CICS transaction manager is permitted to execute the program. Binding in this manner enables DBAs to effectively prohibit programs that are designed to be executed in one environment (e.g., online CICS) from being surreptitiously used in another (e.g., batch).

Auditing

Auditing is a DBMS facility that enables DBAs to track the use of database resources and privileges. When auditing is enabled, the DBMS will produce an audit trail of database operations. Each audited database operation produces an audit trail of information, including what database object was impacted, who performed the operation, and when. Depending on the level of auditing supported by the DBMS, an actual record of what data actually changed may also be recorded. Tracking who does what to which data when is important because there are many threats to the security of your data. (See the “[Threats to Security](#)” sidebar.)

Auditing enables DBAs to track the use of database resources and privileges.

Keep in mind that auditing tracks what a particular user has done once access has been allowed. Auditing occurs post activity; it does not do anything to prohibit access. Audit trails help promote data integrity by enabling the detection of *security breaches*, also referred to as *intrusion detection*. An audited system can serve as a deterrent against users tampering with data because it helps to identify infiltrators.

Auditing occurs post activity; it does not do anything to prohibit access.

Threats to Security

External agents trying to compromise your security and access your company data are rightly viewed as a threat to security. However, industry studies have shown that many security threats are internal—within your organization. The most typical security threat is a disgruntled or malevolent current or ex-employee who has valid access to the DBMS. Auditing is crucial because you may need to find an instance of unauthorized access by an authorized user.

Database auditing is covered in more detail in [Chapter 15](#), “[Regulatory Compliance and Database Administration](#).”

External Security

In addition to database security, the DBA must ensure that certain resources used by the DBMS are protected from being accessed outside the control of the DBMS. If database resources are not accessed using DBMS commands and SQL statements, database security mechanisms cannot be relied on to enforce proper user authentication.

When using external security mechanisms to protect database-related resources, the DBA should focus primarily on the data sets and files used by the DBMS. Data sets to protect at the operating system or file system level include

- System catalog data files
- Active and archive log files
- User data sets for tablespaces
- User data sets for indexes
- Audit data files
- Performance trace files
- Program and script files (both source and executable code)

Focus primarily on the data sets and files used by the DBMS.

Ingenuous users intent on mischief may be able to figure out the format of these files and access unauthorized data if you fail to protect these data sets and files. An additional level of protection can be achieved by compressing the data within the DBMS. This places the additional burden on the hacker of

trying to decompress the data. Of course, compression is not sufficient protection.

If data encryption software is available for use within database files, consider using it. Data encryption is a security technique that encodes legible data into a scrambled format, making the files unreadable without the encryption key. The general idea is to make the effort of decrypting so difficult as to outweigh the advantage to the hacker of accessing the unauthorized data.

Additional security may need to be applied to DBMS system resources, such as the physical storage and address spaces used to run the DBMS, the DBMS console, and files used to install the DBMS.

Job Scheduling and Security

Most organizations schedule tasks to be run at predetermined times, and when those tasks involve database access, authority must be granted to the scheduler. Scheduling is usually accomplished using a third-party job scheduler such as CA-7, Control-M, or AutoSys. When scheduling software is used to control the submission and scheduling of batch programs and scripts, the DBA will have to determine the best way to grant database security to the scheduler.

It is not a very good idea to grant SYSADM authority to the job scheduler. Doing so would permit any job to perform any database task—creating potentially severe security problems. Instead, determine how to grant individual authorization to specific jobs using the facilities of the scheduling package and the DBMS. Many job schedulers can be set up to generate a user ID for each job. The generated ID can be granted the proper authority based on the types of actions that are authorized for that particular job.

It is not a good idea to grant SYSADM authority to the job scheduler.

Another common security mistake made at some shops is embedding actual passwords into database utility jobs and scripts. If the password is hard-coded into the job, anyone can read it and use it elsewhere in the system. This does not protect the security of your data.

Non-DBMS DBA Security

The DBA will need to possess a fairly high level of operating system authority in order to perform the job of administering and managing the organization's databases and data. For example, in the UNIX environment some installation tasks require root authority. This situation can be handled in two ways: Either grant the DBA root authority to do the installation or turn the specific installation tasks requiring root authority over to the UNIX system administrator. Either option is viable. My preference is to grant the authority to the DBAs if the DBA staff possesses the requisite level of UNIX skills to understand the ramifications of having root authority. Either way, though, the DBAs and SAs will need to cooperate in order to create an effective operating system security approach that enables the DBAs to perform their job while at the same time protecting the security and integrity of the platform.

DBMS Fixpacks and Maintenance

Another important consideration with regard to database security is the development and implementation of a DBMS maintenance plan. Like any system software, the DBMS itself must be periodically patched to remedy security defects and other software bugs in the DBMS code. Failing to apply patches as they are delivered from the DBMS vendor via fixpacks, downloads, and other maintenance streams can cause security issues in your databases.

Refer to the sidebar on Oracle Database patches for a discussion of how one vendor, Oracle, manages the software maintenance of its DBMS.

Oracle Database Patches

Oracle has implemented a consistent process for patching its system software, including its DBMS. There are two types of patches for which DBAs need to be prepared: Critical Patch Updates and Security Alerts.

Critical Patch Updates are the primary means of releasing security fixes for Oracle products to customers with valid support contracts. They are released on the Tuesday closest to the seventeenth day of January, April, July, and October.

Security Alerts are issued by Oracle for vulnerability fixes deemed to be too critical to wait for the next Critical Patch Update.

Of course, you should evaluate each patch on a case-by-case basis. You may want to apply critical security patches as soon as they are available. On the other hand, you may want to delay applying patches that are not deemed to be urgent by the DBMS vendor. Remember, when you apply a patch, you are changing your system software and you could be introducing additional flaws and defects. Unless you have the time and resources to conduct a thorough and rigorous test of every patch, only urgent or critical patches should be applied quickly.

Some fixpacks contain literally hundreds of software patches. Although it is a good idea to strive for currency in your DBMS software, use prudent judgment before applying every patch the DBMS vendor supplies to you.

Summary

Database security is an important component of a DBA's job. Without a comprehensive database security plan and implementation, the integrity of your organization's databases will become compromised. DBAs should learn the security mechanisms at their disposal to assure that only authorized users are accessing and changing data in the company's databases.

DBAs should learn the security mechanisms at their disposal.

Furthermore, DBAs should implement auditing operations to verify that the database security measures being deployed are adequate.

Review

1. What is the purpose of a login?
2. Explain two ways that views can be used to implement data security.
3. Once a password has been established for a login, it cannot be changed: true or false?
4. What two SQL DCL statements are used to establish and remove database privileges?
5. If a password needs to be deactivated for a period of time, but may need to be reactivated later, it can be _____ instead of _____.
6. What is a cascading REVOKE and what effect can it have on database security?

7. Is “2926Glenmore” a good password? Why or why not?
8. What is the effect of granting a privilege to PUBLIC?
9. Why is it important to protect some database resources using security mechanisms external to the DBMS?
10. Explain four useful ways of helping to prevent SQL injection attacks.

Suggested Reading

Afyouni, Hassan A. *Database Security and Auditing: Protecting Data Integrity and Accessibility*. Boston, MA: Thomson (2006). ISBN 0-619-21559-3.

Ben Natan, Ron. *Implementing Database Security and Auditing*. Oxford, UK: Elsevier Digital Press (2005). ISBN 978-1-55558-334-7.

Bond, Rebecca, et al. *Understanding DB2 9 Security*. Indianapolis, IN: IBM Press (2007). ISBN 0-13-134590-7.

Castano, Silvana, Mariagrazia Fugini, Giancarlo Martella, and Pierangela Samarati. *Database Security*. Wokingham, England: Addison-Wesley/ACM Press (1995). ISBN 0-201-59375-0.

Clarke, Justin. *SQL Injection Attacks and Defense*. Burlington, MA: Syngress (2009). ISBN 978-1-59749-424-3.

Denning, Dorothy E. *Cryptography and Data Security*. Reading, MA: Addison-Wesley (1982). ISBN 0-201-10150-5.

Ingram, Aaron, and Josh Shaul. *Practical Oracle Security*. Rockland, MA: Syngress (2007). ISBN 978-1-59749-198-3.

Kenan, Kevin. *Cryptography in the Database: The Last Line of Defense*. Boston, MA: Symantec Press/Addison-Wesley (2006). ISBN 0-321-32073-5.

Litchfield, David, et al. *The Database Hacker's Handbook: Defending Database Servers*. Indianapolis, IN: John Wiley & Sons (2005). ISBN 0-7645-7801-4.

Oram, Andy, and John Viega. *Beautiful Security*. Sebastopol, CA: O'Reilly (2009). ISBN 978-0-596-52748-8.

Thuraisingham, Bhavani. *Database and Applications Security: Integrating Information Security and Data Management*. Boca Raton, FL: Auerbach

(2005). ISBN 0-8493-2224-3.